

Laboratório de Concorrência · Aula 10

Matheus Eduardo Polizelli · RA 24129411-2 · Turma A

Gerado em 09/10/2026, 23:00:41 · tempo registrado no cronômetro: 0:00

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

A sincronização de cooperação ocorre quando uma tarefa espera outra finalizar, por exemplo em uma esteira de produção industrial, onde uma etapa depende que a anterior esteja finalizada. Por outro lado a sincronização de competição ocorre quando duas tarefas querem usar o mesmo recurso, mas não pode ser usado ao mesmo tempo, por exemplo se duas pessoas estão tentando comer uma sobremesa e há somente um talher, não há como os dois utilizarem ao mesmo tempo.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

Em teoria, 2 milhões (2.000.000)

1ª execução (n = 1.000.000)

1328614

2ª execução (n = 1.000.000)

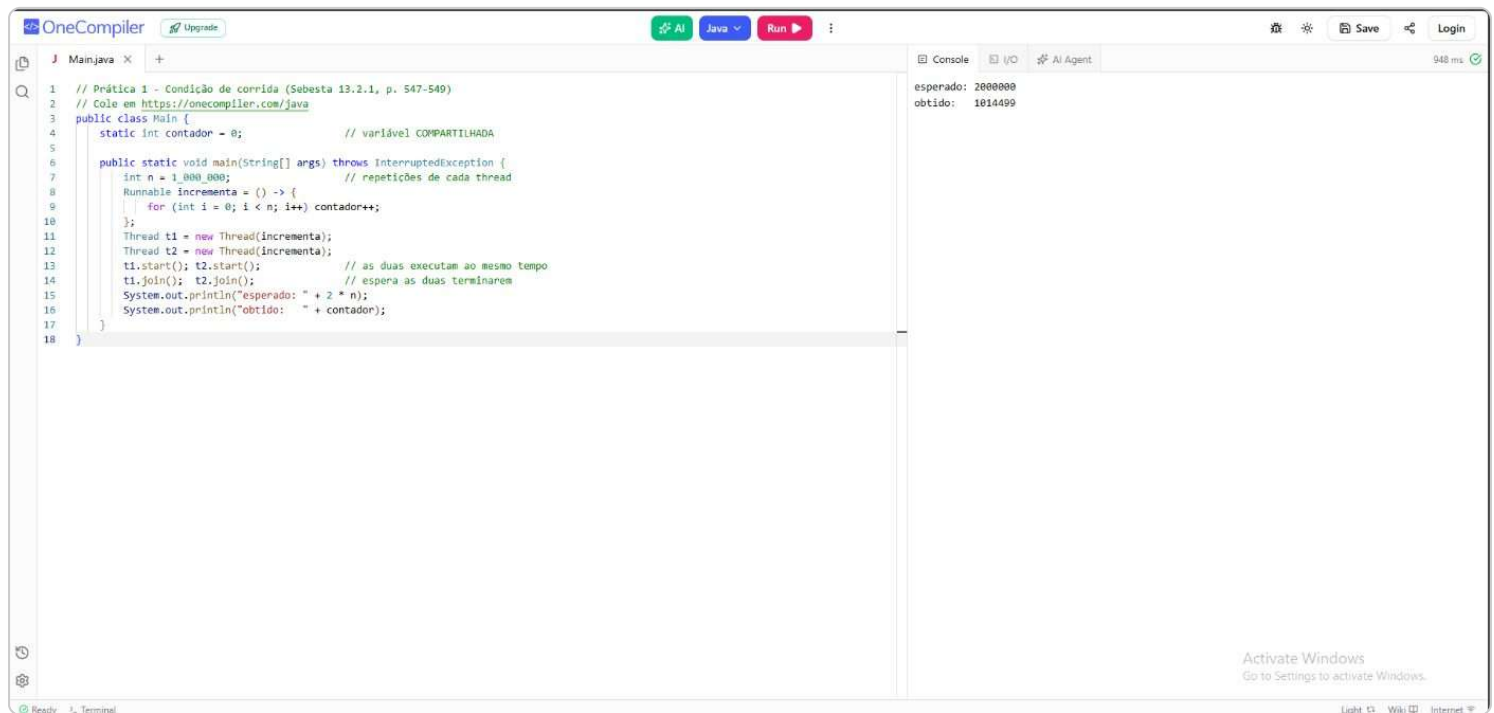
1332674

3ª execução (n = 1.000.000)

1014499

Saída do programa

não respondido



1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Não, foram bem diferentes.

Não, as 3 execuções trouxeram resultados diferentes.

Mudam por conta da imprevisibilidade do escalonador

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Na atualização há 3 passos, ler (verifica o valor na memória), somar (executa a operação matemática) e gravar (salva de fato na memória). A execução de uma thread pode ser interrompida a qualquer momento em favor de outra thread, gerando a race condition. Por exemplo, A fez a leitura e a soma, o escalonador mudou para a thread B, que fez a leitura do que está na memória (sem contar a soma da thread A), realizou a soma e salvou na memória. Quando o escalonador mudar para A novamente, o valor a ser gravado vai ser o mesmo que a thread B salvou.

1.3 Com $n = 1000$, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

com $n = 1000$ o resultado foi 2000, como esperado. Por serem operações extremamente rápidas, provavelmente o escalonamento não foi necessário por isso funcionou como esperado. Em sistemas reais em produção isso é extremamente perigoso, pois é um erro que somente ocorre em cargas que estressam mais o sistema, tal qual a diferença em ambiente de desenvolvimento e produção.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AABBBB → 4

BAAABB → 6

BBBAAA → 7

AAABBB → 8

Saída do programa

não respondido

```

1 # Prática 2 - Todas as respostas possíveis da Figura 13.1 (Sebesta, p. 548)
2 # Cole em https://onecompiler.com/python
3 # TOTAL começa em 3. A tarefa A faz TOTAL + 1 e a tarefa B faz TOTAL * 2.
4 # Cada tarefa tem 3 passos: 1) ler TOTAL 2) calcular 3) gravar TOTAL.
5 from itertools import combinations
6
7
8 def executa(ordem):
9     total = 3
10    reg = {"A": 0, "B": 0}
11    passo = {"A": 0, "B": 0}
12    for t in ordem:
13        if passo[t] == 0:
14            reg[t] = total
15        elif passo[t] == 1:
16            reg[t] = reg[t] + 1 if t == "A" else reg[t] * 2
17        else:
18            total = reg[t]
19        passo[t] += 1
20    return total
21
22
23 print("AAABBB ->", executa("AAABBB")) # A inteira, depois B inteira
24 print("BBBAAA ->", executa("BBBAAA")) # B inteira, depois A inteira
25 print("ABABAB ->", executa("ABABAB")) # alternando passo a passo
26
27 resultados = {}
28 for posicoes in combinations(range(6), 3):
29     ordem = "".join("A" if i in posicoes else "B" for i in range(6))
30     resultados.setdefault(executa(ordem), []).append(ordem)
31
32 print("Ordens possíveis:", sum(len(v) for v in resultados.values()))
33 for valor in sorted(resultados):
34     ordens = resultados[valor]
35     print(f"TOTAL = {valor}: {len(ordens):2d} ordens, por exemplo {ordens[0]}")

```

AAABBB -> 8
BBBAAA -> 7
ABABAB -> 6
Ordens possíveis: 20
TOTAL = 4: 9 ordens, por exemplo AABBBB
TOTAL = 6: 9 ordens, por exemplo AABABB
TOTAL = 7: 1 ordens, por exemplo BBBAAA
TOTAL = 8: 1 ordens, por exemplo AAABBB

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

20 ordens e 4 resultados.

Somente 1 ordem resulta em 8

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

ABBBA resultava em 4

Isso ocorre porque A lê o TOTAL = 3 e o escalonador joga para B. B lê TOTAL = 3, realiza a multiplicação e salva na memória TOTAL = 6. O escalonador volta para A que realiza a soma com TOTAL = 3, pois ele não leu novamente a memória para obter o total novo, por fim salvando 4 no TOTAL.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Acredito que a mais rápida seria a atômica e incorreta seria somente o semáforo caso não seja implementado corretamente

Saída do programa

não respondido

```

1 // Prática 3 - Corrigindo a corrida: monitor e operação atômica (Sebesta 13.4 e 13.7, p. 557-578)
2 // Cole em https://onecompiler.com/java
3 import java.util.concurrent.atomic.AtomicInteger;
4
5 public class Main {
6     static int semCuidado = 0, comSync = 0;
7     static AtomicInteger atomico = new AtomicInteger();
8
9     static synchronized void incSync() { comSync++; } // uma thread por vez
10
11     static long mede(Runnable corpo) throws InterruptedException {
12         Thread a = new Thread(corpo), b = new Thread(corpo);
13         long inicio = System.nanoTime();
14         a.start(); b.start(); a.join(); b.join();
15         return (System.nanoTime() - inicio) / 1_000_000; // milissegundos
16     }
17
18     public static void main(String[] args) throws InterruptedException {
19         int n = 1_000_000;
20         long ms1 = mede(() -> { for (int i = 0; i < n; i++) semCuidado++; });
21         long ms2 = mede(() -> { for (int i = 0; i < n; i++) incSync(); });
22         long ms3 = mede(() -> { for (int i = 0; i < n; i++) atomico.incrementAndGet(); });
23         System.out.println("esperado: " + 2 * n);
24         System.out.println("sem sincronizacao: " + semCuidado + " em " + ms1 + " ms");
25         System.out.println("synchronized: " + comSync + " em " + ms2 + " ms");
26         System.out.println("AtomicInteger: " + atomico.get() + " em " + ms3 + " ms");
27     }
28 }

```

Console Output:

```

esperado: 2000000
sem sincronizacao: 1496576 em 5 ms
synchronized: 2000000 em 84 ms
AtomicInteger: 2000000 em 69 ms

```

```

1 // Prática 3 - Corrigindo a corrida: monitor e operação atômica (Sebesta 13.4 e 13.7, p. 557-578)
2 // Cole em https://onecompiler.com/java
3 import java.util.concurrent.atomic.AtomicInteger;
4
5 public class Main {
6     static int semCuidado = 0, comSync = 0;
7     static AtomicInteger atomico = new AtomicInteger();
8
9     static synchronized void incSync() { comSync++; } // uma thread por vez
10
11     static long mede(Runnable corpo) throws InterruptedException {
12         Thread a = new Thread(corpo), b = new Thread(corpo);
13         long inicio = System.nanoTime();
14         a.start(); b.start(); a.join(); b.join();
15         return (System.nanoTime() - inicio) / 1_000_000; // milissegundos
16     }
17
18     public static void main(String[] args) throws InterruptedException {
19         int n = 1_000_000;
20         long ms1 = mede(() -> { for (int i = 0; i < n; i++) semCuidado++; });
21         long ms2 = mede(() -> { for (int i = 0; i < n; i++) incSync(); });
22         long ms3 = mede(() -> { for (int i = 0; i < n; i++) atomico.incrementAndGet(); });
23         System.out.println("esperado: " + 2 * n);
24         System.out.println("sem sincronizacao: " + semCuidado + " em " + ms1 + " ms");
25         System.out.println("synchronized: " + comSync + " em " + ms2 + " ms");
26         System.out.println("AtomicInteger: " + atomico.get() + " em " + ms3 + " ms");
27     }
28 }

```

Console Output:

```

esperado: 2000000
sem sincronizacao: 1043196 em 2 ms
synchronized: 2000000 em 98 ms
AtomicInteger: 2000000 em 73 ms

```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

As com synchronized ou AtomicInteger. As corretas são mais lentas justamente por lidarem com os problemas de condição de corrida, limitando e gerenciando o acesso ao recurso compartilhado, fazendo com que leve mais tempo.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não funciona pois é uma operação que cuida de apenas uma variável, já a transferência lida com o saldo de quem manda E de quem recebe. Seria preciso usar múltiplos locks ou synchronized

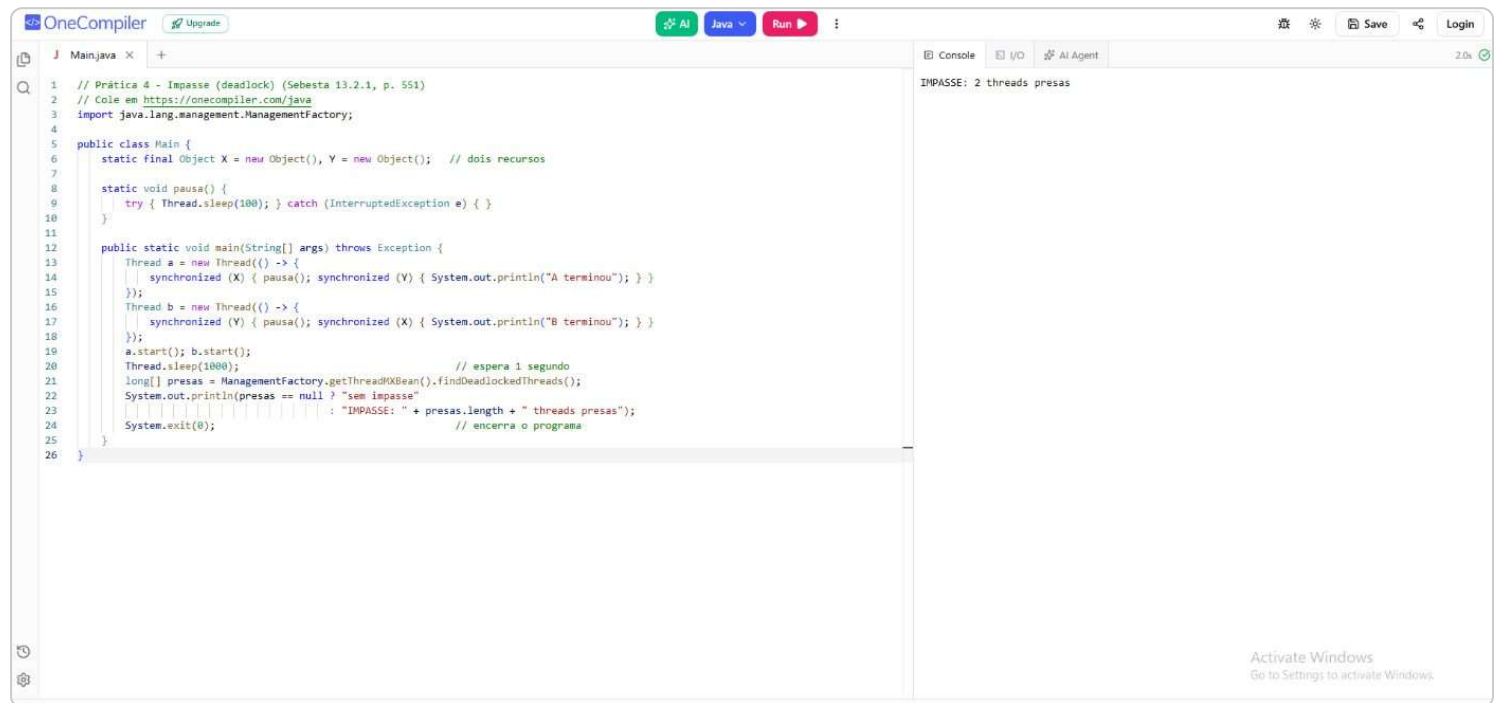
Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Não, justamente por conta do deadlock

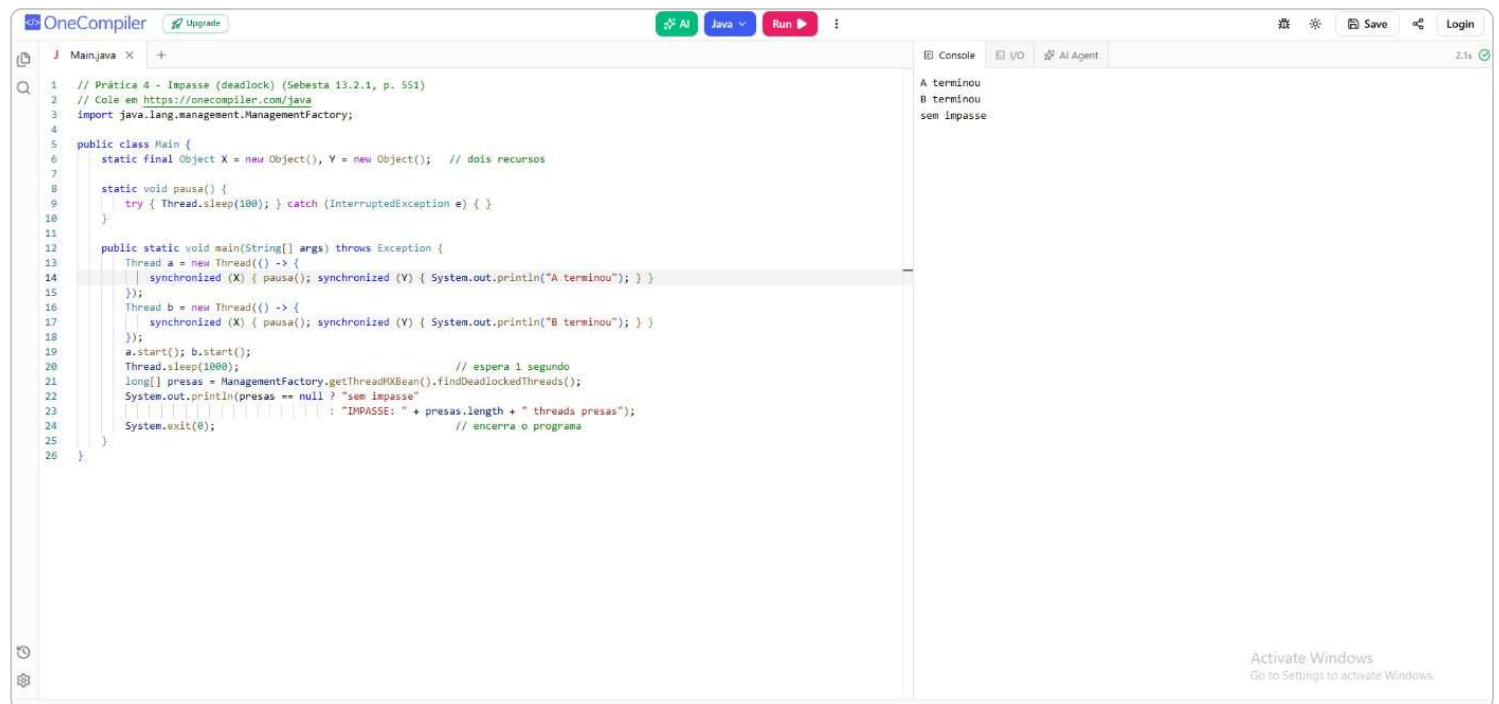
Saídas (antes e depois da correção)

não respondido



```
1 // Prática 4 - Impasse (deadlock) (Sebesta 13.2.1, p. 551)
2 // Cole em https://onecompiler.com/java
3 import java.lang.management.ManagementFactory;
4
5 public class Main {
6     static final Object X = new Object(), Y = new Object(); // dois recursos
7
8     static void pausa() {
9         try { Thread.sleep(100); } catch (InterruptedException e) { }
10    }
11
12    public static void main(String[] args) throws Exception {
13        Thread a = new Thread(() -> {
14            synchronized (X) { pausa(); synchronized (Y) { System.out.println("A terminou"); } }
15        });
16        Thread b = new Thread(() -> {
17            synchronized (Y) { pausa(); synchronized (X) { System.out.println("B terminou"); } }
18        });
19        a.start(); b.start();
20        Thread.sleep(1000); // espera 1 segundo
21        long[] presas = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
22        System.out.println(presas == null ? "sem impasse"
23                           : "IMPASSE: " + presas.length + " threads presas");
24        System.exit(0); // encerra o programa
25    }
26 }
```

IMPASSE: 2 threads presas



```
1 // Prática 4 - Impasse (deadlock) (Sebesta 13.2.1, p. 551)
2 // Cole em https://onecompiler.com/java
3 import java.lang.management.ManagementFactory;
4
5 public class Main {
6     static final Object X = new Object(), Y = new Object(); // dois recursos
7
8     static void pausa() {
9         try { Thread.sleep(100); } catch (InterruptedException e) { }
10    }
11
12    public static void main(String[] args) throws Exception {
13        Thread a = new Thread(() -> {
14            synchronized (X) { pausa(); synchronized (Y) { System.out.println("A terminou"); } }
15        });
16        Thread b = new Thread(() -> {
17            synchronized (X) { pausa(); synchronized (Y) { System.out.println("B terminou"); } }
18        });
19        a.start(); b.start();
20        Thread.sleep(1000); // espera 1 segundo
21        long[] presas = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
22        System.out.println(presas == null ? "sem impasse"
23                           : "IMPASSE: " + presas.length + " threads presas");
24        System.exit(0); // encerra o programa
25    }
26 }
```

A terminou
B terminou
sem impasse

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

A thread A pega o recurso X e espera Y; ao mesmo tempo, B pega Y e espera X. Nenhuma das duas consegue continuar.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Porque quando buscam 2 recursos na mesma ordem, o "caminho" para bloquear o segundo fica livre, evitando o deadlock. Um exemplo real é a transação financeira do exemplo anterior.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

200ms

600ms

100ms

Tempo com 1 trabalhador

600ms

Tempo com 3 trabalhadores

200ms

Tempo com 6 trabalhadores

100ms

Saída do programa

não respondido

```
1 // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 559-560)
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import (
6     "fmt"
7     "sync"
8     "time"
9 )
10
11 const trabalhadores = 3 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {
30         tarefas <- t // envia uma mensagem
31     }
32     close(tarefas)
33     wg.Wait() // espera todos os trabalhadores
34     fmt.Println(trabalhadores, "trabalhadores, 6 tarefas de 100 ms: levou",
35         time.Since(inicio).Round(10*time.Millisecond))
36 }
```

trabalhador 3 fez a tarefa 1
trabalhador 2 fez a tarefa 3
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 6
trabalhador 3 fez a tarefa 4
trabalhador 2 fez a tarefa 5
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms

```

1 // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 559-560)
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import (
6     "fmt"
7     "sync"
8     "time"
9 )
10
11 const trabalhadores = 1 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {
30         tarefas <- t // envia uma mensagem
31     }
32     close(tarefas)
33     wg.Wait() // espera todos os trabalhadores
34     fmt.Println(trabalhadores, "trabalhadores, 6 tarefas de 100 ms: levou",
35         time.Since(inicio).Round(10*time.Millisecond))
36 }

```

trabalhador 1 fez a tarefa 1
trabalhador 1 fez a tarefa 2
trabalhador 1 fez a tarefa 3
trabalhador 1 fez a tarefa 4
trabalhador 1 fez a tarefa 5
trabalhador 1 fez a tarefa 6
1 trabalhadores, 6 tarefas de 100 ms: levou 600ms

```

1 // Prática 5a - Passagem de mensagens com canais: grupo de trabalhadores (Sebesta 13.5, p. 559-560)
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import (
6     "fmt"
7     "sync"
8     "time"
9 )
10
11 const trabalhadores = 6 // mude para 1 e depois para 6
12
13 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
14     defer wg.Done()
15     for t := range tarefas { // recebe tarefas até o canal ser fechado
16         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
17         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
18     }
19 }
20
21 func main() {
22     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
23     var wg sync.WaitGroup
24     inicio := time.Now()
25     for id := 1; id <= trabalhadores; id++ {
26         wg.Add(1)
27         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
28     }
29     for t := 1; t <= 6; t++ {
30         tarefas <- t // envia uma mensagem
31     }
32     close(tarefas)
33     wg.Wait() // espera todos os trabalhadores
34     fmt.Println(trabalhadores, "trabalhadores, 6 tarefas de 100 ms: levou",
35         time.Since(inicio).Round(10*time.Millisecond))
36 }

```

trabalhador 2 fez a tarefa 3
trabalhador 1 fez a tarefa 4
trabalhador 3 fez a tarefa 2
trabalhador 6 fez a tarefa 1
trabalhador 5 fez a tarefa 6
trabalhador 4 fez a tarefa 5
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Porque não há acesso a recurso compartilhado entre threads

Saídas (antes e depois da troca)

não respondido

The screenshot shows the OneCompiler interface with a Go program in the editor. The program is a simple channel-based test. The console on the right displays a fatal error: "fatal error: all goroutines are asleep - deadlock!". The error details show that goroutine 1 is stuck in a send operation on a channel that no one is receiving from.

```

1 // Prática 5b - Um canal sem ninguém do outro lado
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import "fmt"
6
7 func main() {
8     ch := make(chan int)           // canal SEM espaço (sem buffer)
9     ch <- 1                       // envia... e espera alguém receber
10    fmt.Println(<-ch)
11 }

```

```

fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
    /box/main.go:9 +0x38
exit status 2

```

The screenshot shows the same Go program as the first image, but now it runs successfully. The console on the right shows the output "1", indicating that the channel-based communication worked because the channel had a buffer of size 1.

```

1 // Prática 5b - Um canal sem ninguém do outro lado
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import "fmt"
6
7 func main() {
8     ch := make(chan int,1)        // canal SEM espaço (sem buffer)
9     ch <- 1                       // envia... e espera alguém receber
10    fmt.Println(<-ch)
11 }

```

```

1

```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

No primeiro não há espaço para uma mensagem, que é enviada ao canal, portanto o programa trava pois não tem quem receba a mensagem. Já no segundo há um buffer para uma mensagem, funcionando normalmente

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Ele muda com sucesso a variável

Saídas (o erro e o resultado com move)

não respondido

The image shows the OneCompiler web-based IDE. The top bar includes the OneCompiler logo, an 'Upgrade' button, and navigation icons for AI, Rust, Run, and a settings icon. On the right, there are icons for search, settings, save, and a login button.

The main editor area is split into two panes. The left pane shows a Rust file named 'HelloWorld.rs' with the following code:

```
1 // Prática 6a - Rust: tentando compartilhar uma variável entre threads
2 // Cole em https://onecompiler.com/rust
3 use std::thread;
4
5 fn main() {
6     let mut contador = 0;
7     let t = thread::spawn(|| {
8         contador += 1;           // a thread altera a variável de main
9     });
10    t.join().unwrap();
11    println!("{}", contador);
12 }
```

The right pane shows the console output. It starts with a note about static lifetimes and then shows the compilation error:

```
error[E0502]: cannot borrow `contador` as immutable because it is also borrowed as mutable
--> HelloWorld.rs:11:20
7 |         let t = thread::spawn(|| {
8 |             |
9 |             |
10 |             |
11 |             |
12 |             |
13 |             |
14 |             |
15 |             |
16 |             |
17 |             |
18 |             |
19 |             |
20 |             |
21 |             |
22 |             |
23 |             |
24 |             |
25 |             |
26 |             |
27 |             |
28 |             |
29 |             |
30 |             |
31 |             |
32 |             |
33 |             |
34 |             |
35 |             |
36 |             |
37 |             |
38 |             |
39 |             |
40 |             |
41 |             |
42 |             |
43 |             |
44 |             |
45 |             |
46 |             |
47 |             |
48 |             |
49 |             |
50 |             |
51 |             |
52 |             |
53 |             |
54 |             |
55 |             |
56 |             |
57 |             |
58 |             |
59 |             |
60 |             |
61 |             |
62 |             |
63 |             |
64 |             |
65 |             |
66 |             |
67 |             |
68 |             |
69 |             |
70 |             |
71 |             |
72 |             |
73 |             |
74 |             |
75 |             |
76 |             |
77 |             |
78 |             |
79 |             |
80 |             |
81 |             |
82 |             |
83 |             |
84 |             |
85 |             |
86 |             |
87 |             |
88 |             |
89 |             |
90 |             |
91 |             |
92 |             |
93 |             |
94 |             |
95 |             |
96 |             |
97 |             |
98 |             |
99 |             |
100 |             |
101 |             |
102 |             |
103 |             |
104 |             |
105 |             |
106 |             |
107 |             |
108 |             |
109 |             |
110 |             |
111 |             |
112 |             |
113 |             |
114 |             |
115 |             |
116 |             |
117 |             |
118 |             |
119 |             |
120 |             |
121 |             |
122 |             |
123 |             |
124 |             |
125 |             |
126 |             |
127 |             |
128 |             |
129 |             |
130 |             |
131 |             |
132 |             |
133 |             |
134 |             |
135 |             |
136 |             |
137 |             |
138 |             |
139 |             |
140 |             |
141 |             |
142 |             |
143 |             |
144 |             |
145 |             |
146 |             |
147 |             |
148 |             |
149 |             |
150 |             |
151 |             |
152 |             |
153 |             |
154 |             |
155 |             |
156 |             |
157 |             |
158 |             |
159 |             |
160 |             |
161 |             |
162 |             |
163 |             |
164 |             |
165 |             |
166 |             |
167 |             |
168 |             |
169 |             |
170 |             |
171 |             |
172 |             |
173 |             |
174 |             |
175 |             |
176 |             |
177 |             |
178 |             |
179 |             |
180 |             |
181 |             |
182 |             |
183 |             |
184 |             |
185 |             |
186 |             |
187 |             |
188 |             |
189 |             |
190 |             |
191 |             |
192 |             |
193 |             |
194 |             |
195 |             |
196 |             |
197 |             |
198 |             |
199 |             |
200 |             |
201 |             |
202 |             |
203 |             |
204 |             |
205 |             |
206 |             |
207 |             |
208 |             |
209 |             |
210 |             |
211 |             |
212 |             |
213 |             |
214 |             |
215 |             |
216 |             |
217 |             |
218 |             |
219 |             |
220 |             |
221 |             |
222 |             |
223 |             |
224 |             |
225 |             |
226 |             |
227 |             |
228 |             |
229 |             |
230 |             |
231 |             |
232 |             |
233 |             |
234 |             |
235 |             |
236 |             |
237 |             |
238 |             |
239 |             |
240 |             |
241 |             |
242 |             |
243 |             |
244 |             |
245 |             |
246 |             |
247 |             |
248 |             |
249 |             |
250 |             |
251 |             |
252 |             |
253 |             |
254 |             |
255 |             |
256 |             |
257 |             |
258 |             |
259 |             |
260 |             |
261 |             |
262 |             |
263 |             |
264 |             |
265 |             |
266 |             |
267 |             |
268 |             |
269 |             |
270 |             |
271 |             |
272 |             |
273 |             |
274 |             |
275 |             |
276 |             |
277 |             |
278 |             |
279 |             |
280 |             |
281 |             |
282 |             |
283 |             |
284 |             |
285 |             |
286 |             |
287 |             |
288 |             |
289 |             |
290 |             |
291 |             |
292 |             |
293 |             |
294 |             |
295 |             |
296 |             |
297 |             |
298 |             |
299 |             |
300 |             |
301 |             |
302 |             |
303 |             |
304 |             |
305 |             |
306 |             |
307 |             |
308 |             |
309 |             |
310 |             |
311 |             |
312 |             |
313 |             |
314 |             |
315 |             |
316 |             |
317 |             |
318 |             |
319 |             |
320 |             |
321 |             |
322 |             |
323 |             |
324 |             |
325 |             |
326 |             |
327 |             |
328 |             |
329 |             |
330 |             |
331 |             |
332 |             |
333 |             |
334 |             |
335 |             |
336 |             |
337 |             |
338 |             |
339 |             |
340 |             |
341 |             |
342 |             |
343 |             |
344 |             |
345 |             |
346 |             |
347 |             |
348 |             |
349 |             |
350 |             |
351 |             |
352 |             |
353 |             |
354 |             |
355 |             |
356 |             |
357 |             |
358 |             |
359 |             |
360 |             |
361 |             |
362 |             |
363 |             |
364 |             |
365 |             |
366 |             |
367 |             |
368 |             |
369 |             |
370 |             |
371 |             |
372 |             |
373 |             |
374 |             |
375 |             |
376 |             |
377 |             |
378 |             |
379 |             |
380 |             |
381 |             |
382 |             |
383 |             |
384 |             |
385 |             |
386 |             |
387 |             |
388 |             |
389 |             |
390 |             |
391 |             |
392 |             |
393 |             |
394 |             |
395 |             |
396 |             |
397 |             |
398 |             |
399 |             |
400 |             |
401 |             |
402 |             |
403 |             |
404 |             |
405 |             |
406 |             |
407 |             |
408 |             |
409 |             |
410 |             |
411 |             |
412 |             |
413 |             |
414 |             |
415 |             |
416 |             |
417 |             |
418 |             |
419 |             |
420 |             |
421 |             |
422 |             |
423 |             |
424 |             |
425 |             |
426 |             |
427 |             |
428 |             |
429 |             |
430 |             |
431 |             |
432 |             |
433 |             |
434 |             |
435 |             |
436 |             |
437 |             |
438 |             |
439 |             |
440 |             |
441 |             |
442 |             |
443 |             |
444 |             |
445 |             |
446 |             |
447 |             |
448 |             |
449 |             |
450 |             |
451 |             |
452 |             |
453 |             |
454 |             |
455 |             |
456 |             |
457 |             |
458 |             |
459 |             |
460 |             |
461 |             |
462 |             |
463 |             |
464 |             |
465 |             |
466 |             |
467 |             |
468 |             |
469 |             |
470 |             |
471 |             |
472 |             |
473 |             |
474 |             |
475 |             |
476 |             |
477 |             |
478 |             |
479 |             |
480 |             |
481 |             |
482 |             |
483 |             |
484 |             |
485 |             |
486 |             |
487 |             |
488 |             |
489 |             |
490 |             |
491 |             |
492 |             |
493 |             |
494 |             |
495 |             |
496 |             |
497 |             |
498 |             |
499 |             |
500 |             |
501 |             |
502 |             |
503 |             |
504 |             |
505 |             |
506 |             |
507 |             |
508 |             |
509 |             |
510 |             |
511 |             |
512 |             |
513 |             |
514 |             |
515 |             |
516 |             |
517 |             |
518 |             |
519 |             |
520 |             |
521 |             |
522 |             |
523 |             |
524 |             |
525 |             |
526 |             |
527 |             |
528 |             |
529 |             |
530 |             |
531 |             |
532 |             |
533 |             |
534 |             |
535 |             |
536 |             |
537 |             |
538 |             |
539 |             |
540 |             |
541 |             |
542 |             |
543 |             |
544 |             |
545 |             |
546 |             |
547 |             |
548 |             |
549 |             |
550 |             |
551 |             |
552 |             |
553 |             |
554 |             |
555 |             |
556 |             |
557 |             |
558 |             |
559 |             |

```

The image shows a screenshot of the OneCompiler web-based IDE. The interface is divided into three main sections: a top toolbar, a left sidebar, and a main workspace.

Top Toolbar: Includes the OneCompiler logo, an "Upgrade" button, and buttons for "AI", "Rust", "Run", and "Login".

Left Sidebar: Contains a file explorer showing a single file named "HelloWorld.rs".

Main Workspace: The editor displays the following Rust code:

```
1 // Prática 6a - Rust: tentando compartilhar uma variável entre threads
2 // Cole em https://onecompiler.com/rust
3 use std::thread;
4
5 fn main() {
6     let mut contador = 0;
7     let t = thread::spawn(move || {
8         contador += 1; // a thread altera a variável de main
9     });
10    t.join().unwrap();
11    println!("{}", contador);
12 }
```

Below the code editor, the "Console" tab is active, showing the following output:

```
warning: value captured by 'contador' is never read
--> HelloWorld.rs:8:9
|
8 |         contador += 1; // a thread altera a variável de main
|         ^^^^^^^^^
|
= help: did you mean to capture by reference instead?
= note: '#[warn(unused_assignments)]' (part of '#[warn(unused)]') on by default

warning: value assigned to 'contador' is never read
--> HelloWorld.rs:8:9
|
8 |         contador += 1; // a thread altera a variável de main
|         ^^^^^^^^^
|
= help: maybe it is overwritten before being read

warning: 2 warnings emitted

0
```

At the bottom right of the screen, there is a watermark that says "Activate Windows Go to Settings to activate Windows."

Saída do programa

não respondido

```

1 // Prática 0b - Rust: compartilhar com Arc (contagem de referências) e Mutex (exclusão mútua)
2 // Cole em https://onecompiler.com/rust
3 use std::sync::{Arc, Mutex};
4 use std::thread;
5
6 fn main() {
7     let contador = Arc::new(Mutex::new(0)); // compartilhado e protegido
8     let mut threads = Vec::new();
9     for _ in 0..4 {
10         let c = Arc::clone(&contador);
11         threads.push(thread::spawn(move || {
12             for _ in 0..100_000 {
13                 *c.lock().unwrap() += 1; // trava, soma e destrava
14             }
15         }));
16     }
17     for t in threads { t.join().unwrap(); }
18     println!("contador = {}", *contador.lock().unwrap());
19 }

```

contador = 400000

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Por levar só uma cópia do número, ao incrementá-lo o valor do número original não é alterado, portanto imprime 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Rust é descoberto em tempo de compilação, já em Java em tempo de execução. Em tempo de compilação no geral é melhor pois o erro é descoberto antes de sequer acontecer. Ainda que leve mais tempo para compilar, economiza custos de manutenção e evita problemas em produção, possivelmente catastróficos.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

ADCB

Saída do programa

não respondido

```

A
D
C
B
E: timer de 10 ms rodou depois de 300 ms

```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

ADCB.

B saiu depois de D (que é síncrono, tem preferência de execução) e C (é colocado na fila de microtasks) pois a fila de macrotasks que setTimeout tem menor prioridade.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

o código síncrono "ocupado(300)" é executado antes do setTimeout (macrotask), por isso só executa depois dos 300ms. Isso, caso o usuário não conheça o funcionamento do event loop, é fácil sobrecarregar o servidor ou determinada rotina "sequestrar" todo recurso e travar o event loop

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode ocorrer uma condição de corrida e dois usuários tentarem comprar o mesmo ingresso ao mesmo tempo. Para evitar utilizaria uma técnica como Semáforo para bloquear o recurso compartilhado enquanto ele estiver sendo usado, evitando a race condition

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

Eu já conhecia concorrência, mas não tão a fundo e nem como funcionava em múltiplas linguagens!